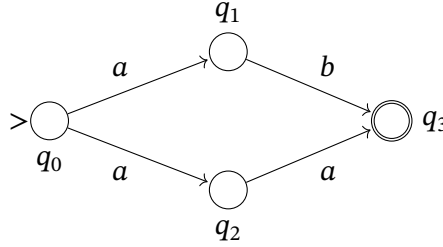


Finite Automata and Regular Expressions

Q1. Let N be the NFA below over $\{a, b\}$.



- Show that $L(N) = \{aa, ab\}$.
- Obtain an equivalent DFA for N .
- One state of your DFA contains both q_1 and q_2 . In Tutorial 3, a DFA state grouping multiple NFA states always came from an ϵ -closure. What puts them together here?

Theorem (Closure properties of automaton languages)

The class of languages accepted by finite automata is closed under union, concatenation, Kleene star, complementation, and intersection.

Let $M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$ and $M_2 = (K_2, \Sigma, \Delta_2, s_2, F_2)$ be NFAs with $K_1 \cap K_2 = \emptyset$.

Union. $M = (K_1 \cup K_2 \cup \{s\}, \Sigma, \Delta, s, F_1 \cup F_2)$ accepts $L(M_1) \cup L(M_2)$, where $s \notin K_1 \cup K_2$ and $\Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2)\}$.

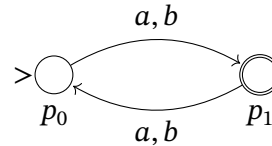
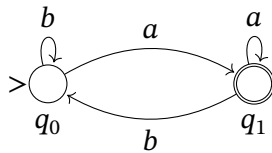
Concatenation. $M = (K_1 \cup K_2, \Sigma, \Delta, s_1, F_2)$ with $\Delta = \Delta_1 \cup \Delta_2 \cup \{(q, e, s_2) : q \in F_1\}$ accepts $L(M_1) \circ L(M_2)$.

Kleene star. $M = (K_1 \cup \{s\}, \Sigma, \Delta, s, F_1 \cup \{s\})$ accepts $L(M_1)^*$, where $s \notin K_1$ and $\Delta = \Delta_1 \cup \{(s, e, s_1)\} \cup \{(q, e, s_1) : q \in F_1\}$.

Complementation. Let $M = (K, \Sigma, \delta, s, F)$ be a DFA; then $M' = (K, \Sigma, \delta, s, K - F)$ accepts $\Sigma^* - L(M)$.

Intersection. Since $L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$, closure under intersection follows from closure under union and complementation.

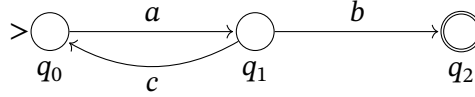
Q2. Let M_1 and M_2 be the NFAs below over $\{a, b\}$.



- Using the union construction above, draw an NFA M accepting $L(M_1) \cup L(M_2)$.
- Using the concatenation construction above, draw an NFA M accepting $L(M_1) \circ L(M_2)$.
- Describe $L(M_1)$ and $L(M_2)$ in plain English, and use this to describe $L(M_1) \circ L(M_2)$ in plain English.

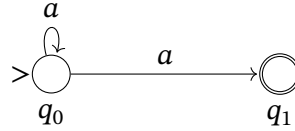
Q3. The Kleene star construction above adds a new state and makes it the initial state. Suppose instead we tried to save a state: leave s_1 as the start state, make it a final state, and add an ϵ -transition (q, ϵ, s_1) for every $q \in F_1$.

Let M_1 be the NFA below over $\{a, b, c\}$.



- Apply the modified construction above (without a new start state) to M_1 , and give the resulting automaton M .
- Show that $ac \in L(M)$, despite $ac \notin L(M_1)^*$.
- In a sentence or two, explain in general why omitting the new state s can cause this kind of error.

Q4. Let N be the NFA below over $\{a\}$.



- Let N' be obtained from N by swapping final and nonfinal states, keeping the same transitions. Find $L(N')$. Is $L(N') = \Sigma^* - L(N)$?
- Why does interchanging final and nonfinal states for complementation only work on DFAs?

Theorem

A language is regular if and only if it is accepted by some finite automaton.

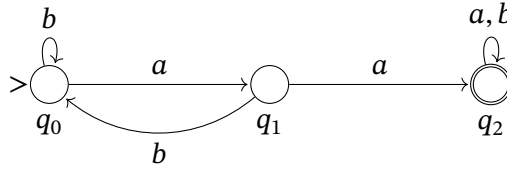
Q5. Using the closure properties' constructions above, create a finite automaton accepting $\mathcal{L}((ab \cup ba)^*a)$. At each stage, draw the automaton for the sub-expression built so far.

Algorithm Converting a finite automaton to a regular expression

Require: automaton M with unique start state s , unique final state f , no edges into s , none out of f

- Merge parallel edges between the same pair of states into one, labeled by their union
- while** M has a state $q \notin \{s, f\}$ **do**
- Let γ be the label of the self-loop at q (or $\emptyset$ if none)
- for all** $q_i \xrightarrow{\alpha} q \xrightarrow{\beta} q_j$ with $q_i, q_j \neq q$ **do**
- if** edge $q_i \rightarrow q_j$ exists, labeled δ : relabel it $\delta \cup \alpha\gamma^*\beta$
- else** add edge $q_i \rightarrow q_j$ labeled $\alpha\gamma^*\beta$
- end for**
- Remove q and all its incident edges
- end while**
- return** the label on the edge from s to f (or $\emptyset$ if none)

Q6. Let M be the DFA below over $\{a, b\}$, accepting the language of all strings containing aa as a substring.



- M is not yet in the form required by the algorithm above. Modify M to bring it into this form.
- Run the algorithm above on M , eliminating states one at a time, to obtain a regular expression for $L(M)$. Show the automaton after each elimination.
- Verify that your regular expression matches $baab$ and does not match $abab$.

Q7. Let $L \subseteq \Sigma^*$. Define $\text{Pref}(L) = \{w \in \Sigma^* : wy \in L \text{ for some } y \in \Sigma^*\}$, the set of all prefixes of strings in L .

- Let $M = (K, \Sigma, \Delta, s, F)$ be an NFA accepting L . Define

$$F' = \{q \in K : (q, w) \vdash_M^* (f, e) \text{ for some } w \in \Sigma^*, f \in F\},$$

the set of states from which some final state is reachable. Prove that the NFA $M' = (K, \Sigma, \Delta, s, F')$ accepts $\text{Pref}(L)$.

- Apply this to the NFA M_1 from Q2 to describe $\text{Pref}(L(M_1))$ in plain English.

Q8. Let $M = (K, \Sigma, \Delta, s, F)$ be an NFA accepting L . Define $M^R = (K \cup \{s'\}, \Sigma, \Delta^R, s', \{s\})$, where $s' \notin K$ and

$$\Delta^R = \{(p, a, q) : (q, a, p) \in \Delta\} \cup \{(s', e, f) : f \in F\}.$$

- Prove that $L(M^R) = L^R$, where $L^R = \{w^R : w \in L\}$. Conclude that regular languages are closed under reversal.
- Apply this construction to M_1 from Q2 to obtain M_1^R , and describe $L(M_1^R)$ in plain English.
- In Tutorial 2, Q4, you constructed a regular expression for $\mathcal{L}(\alpha)^R$ directly from a regular expression α . Which of the two approaches, reversing a regular expression or reversing an automaton, gives you more insight into why reversal preserves regularity?